

BLM3522 BULUT BİLİŞİM DERSİ

PROJE 2 RAPORU

Gerçek Zamanlı IoT Veri Akışı Sistemi

Hazırlayan	Yağmur Kalyoncu -23291413
Ders	BLM3522 Bulut Bilişim ve Uygulamaları
Öğretim Görevlisi	Enver Bağcı
Proje	Gerçek Zamanlı IoT Veri Akışı Sistemi
GitHub	https://github.com/Yagmurkalyoncu/Bulut-Bilisim-Proje-2
Youtube	https://youtu.be/9Bd3GAjUrng?si=MG7OdaoONHoO_qOA

1. Proje Amacı

Bu projede, gerçek zamanlı IoT (Nesnelerin İnterneti) veri akışı simüle edilmiş ve bulut platformunda işlenmiştir. Ankara'nın dört farklı ilçesinde (Merkez, Çankaya, Keçiören, Gölbaşı) konumlandırılmış sanal sensörlerden sıcaklık, nem, basınç ve CO2 verileri üretilmiştir.

Veriler WebSocket protokolü üzerinden toplanmış, AWS Kinesis Data Streams aracılığıyla buluta iletilmiş, AWS Lambda fonksiyonu ile işlenmiş ve AWS DynamoDB'ye kaydedilmiştir. Streamlit tabanlı gerçek zamanlı bir dashboard ile verilerin anlık görselleştirilmesi sağlanmıştır.

Proje boyunca WebSocket protokolü, AWS bulut servisleri, zaman serisi veri yönetimi ve Git versiyon kontrol sistemi gibi teknolojiler uygulamalı olarak öğrenilmiş ve kullanılmıştır.

ÖZELLİKLER

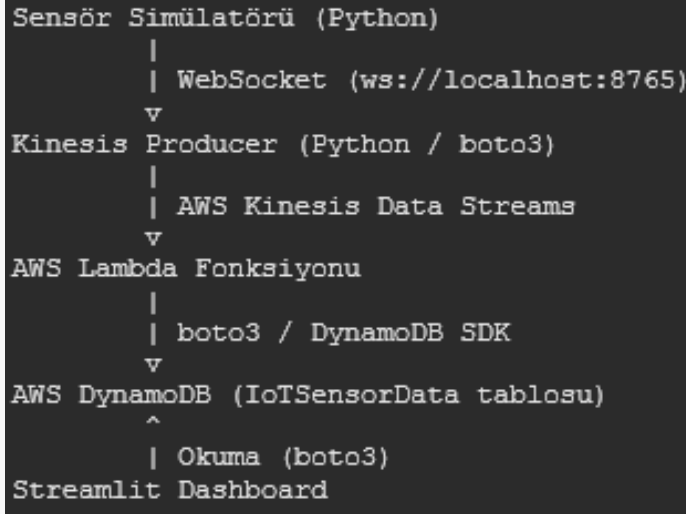
- 4 farklı Ankara lokasyonundan gerçek zamanlı sensör verisi simülasyonu
- WebSocket sunucusu ile anlık veri yayını (her 2 saniyede bir)
- AWS Kinesis Data Streams ile yüksek hacimli veri akışı yönetimi
- AWS Lambda ile sunucusuz veri işleme ve anomali tespiti
- AWS DynamoDB ile zaman serisi veri depolama
- Streamlit ile interaktif gerçek zamanlı dashboard

2. Kullanılan Teknolojiler

Katman / Teknoloji	Araç	Açıklama
Backend Dili	Python 3.11	Tüm bileşenler Python ile yazıldı
Protokol	WebSocket (websockets kütüphanesi)	Gerçek zamanlı çift yönlü iletişim
Alternatif Protokol	MQTT (paho-mqtt, HiveMQ)	IoT standart publish/subscribe modeli
Veri Akışı	AWS Kinesis Data Streams	Yüksek hacimli gerçek zamanlı veri akışı
Sunucusuz İşleme	AWS Lambda	Kinesis tetiklemeli otomatik veri işleme
Veritabanı	AWS DynamoDB	Zaman serisi veri depolama (NoSQL)
Dashboard	Streamlit + Plotly	Gerçek zamanlı interaktif görselleştirme
Bulut Platformu	AWS (us-east-1)	Kinesis, Lambda, DynamoDB, IAM, CloudWatch
Versiyon Kontrol	Git + GitHub	Kod yönetimi ve commit geçmişi

2.1 Mimari Tasarım

Sistem, olay güdümlü (event-driven) ve sunucusuz (serverless) bir mimari üzerine inşa edilmiştir. Bileşenler birbirinden bağımsız çalışır ve AWS yönetilen servisleri aracılığıyla haberleşir.



3. Sensör Simülatörü ve WebSocket Sunucusu

sensor_simulator.py dosyası, gerçek IoT cihazlarını taklit eden bir WebSocket sunucusu olarak çalışır.

Ankara'nın dört ilçesinde konumlandırılmış sanal sensörler, Python asyncio kütüphanesi ile asenkron olarak veri üretir

3.1 Sensör Lokasyonları

Sensör ID	Lokasyon
sensor-ankara-01	Ankara-Merkez
sensor-ankara-02	Ankara-Çankaya
sensor-ankara-03	Ankara-Keçiören
sensor-ankara-04	Ankara-Gölbaşı

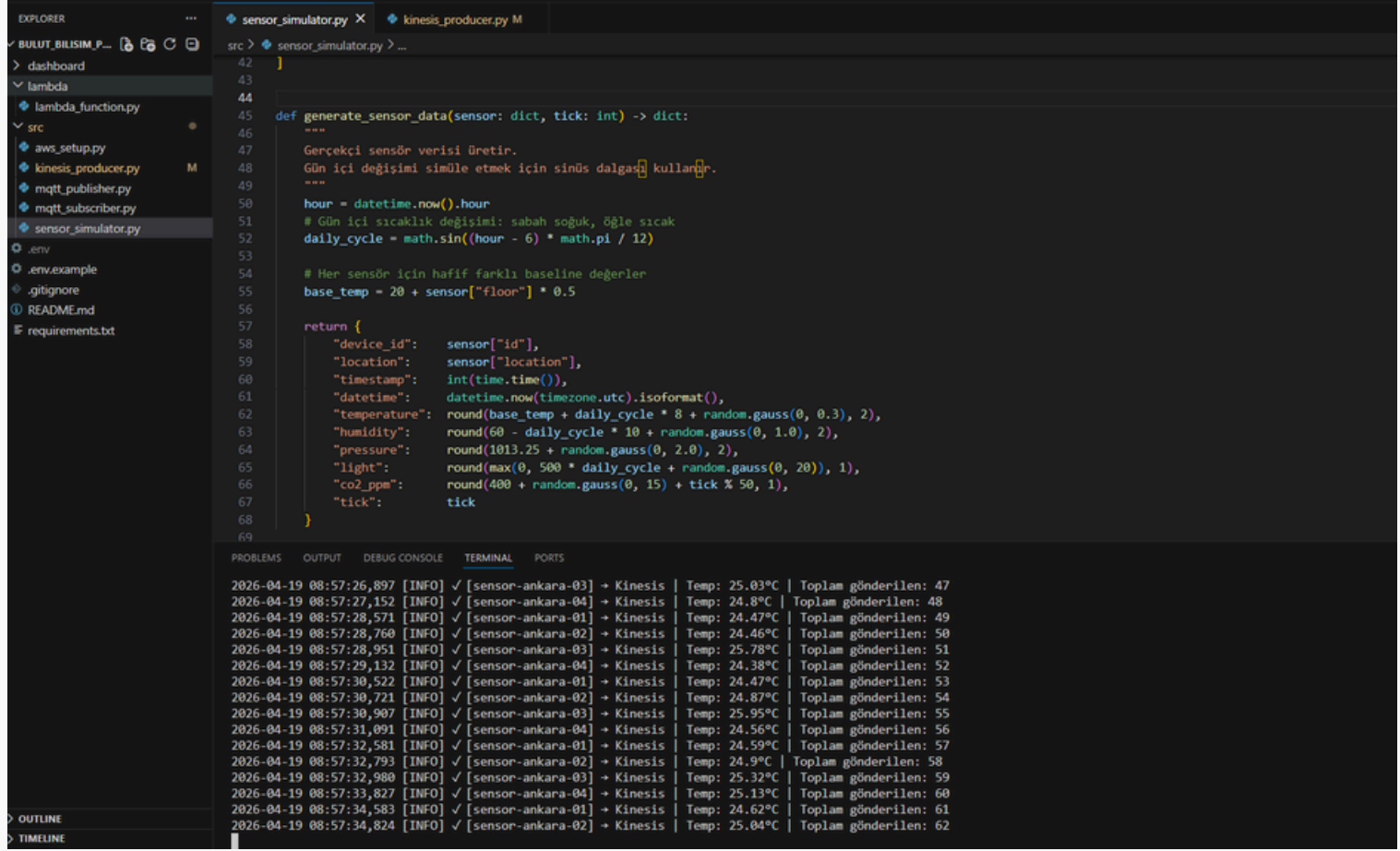
3.2 Üretilen Veri Formatı

Her sensörden üretilen JSON veri paketi aşağıdaki alanları içermektedir:

```
{
  "device_id": "sensor-ankara-01",
  "location": "Ankara-Merkez",
  "timestamp": 1776514983,
  "datetime": "2026-04-18T12:23:06+00:00",
  "temperature": 26.43,
  "humidity": 53.55,
  "pressure": 1013.84,
  "light": 320.0,
  "co2_ppm": 392.1
}
```

3.3 Gerçekçi Veri Üretimi

Simülör, gerçek dünya koşullarını yansıtmak için matematiksel modeller kullanır. Gün içi sıcaklık değişimi sinüs dalgası ile modellenir; sabah saatlerinde düşük, öğle saatlerinde yüksek sıcaklık değerleri üretilir. Gaussian gürültü (random.gauss) eklenerek sensör titremesi simüle edilir.



```
42 }
43
44
45 def generate_sensor_data(sensor: dict, tick: int) -> dict:
46     """
47     Gerçekçi sensör verisi üretir.
48     Gün içi değişimi simüle etmek için sinüs dalgası kullanır.
49     """
50     hour = datetime.now().hour
51     # Gün içi sıcaklık değişimi: sabah soğuk, öğle sıcak
52     daily_cycle = math.sin((hour - 6) * math.pi / 12)
53
54     # Her sensör için hafif farklı baseline değerler
55     base_temp = 20 + sensor["floor"] * 0.5
56
57     return {
58         "device_id": sensor["id"],
59         "location": sensor["location"],
60         "timestamp": int(time.time()),
61         "datetime": datetime.now(timezone.utc).isoformat(),
62         "temperature": round(base_temp + daily_cycle * 8 + random.gauss(0, 0.3), 2),
63         "humidity": round(60 - daily_cycle * 10 + random.gauss(0, 1.0), 2),
64         "pressure": round(1013.25 + random.gauss(0, 2.0), 2),
65         "light": round(max(0, 500 * daily_cycle + random.gauss(0, 20)), 1),
66         "co2_ppm": round(400 + random.gauss(0, 15) + tick % 50, 1),
67         "tick": tick
68     }
69
```

2026-04-19 08:57:26,897 [INFO] ✓ [sensor-ankara-03] → Kinesis | Temp: 25.03°C | Toplam gönderilen: 47
2026-04-19 08:57:27,152 [INFO] ✓ [sensor-ankara-04] → Kinesis | Temp: 24.8°C | Toplam gönderilen: 48
2026-04-19 08:57:28,571 [INFO] ✓ [sensor-ankara-01] → Kinesis | Temp: 24.47°C | Toplam gönderilen: 49
2026-04-19 08:57:28,760 [INFO] ✓ [sensor-ankara-02] → Kinesis | Temp: 24.46°C | Toplam gönderilen: 50
2026-04-19 08:57:28,951 [INFO] ✓ [sensor-ankara-03] → Kinesis | Temp: 25.78°C | Toplam gönderilen: 51
2026-04-19 08:57:29,132 [INFO] ✓ [sensor-ankara-04] → Kinesis | Temp: 24.38°C | Toplam gönderilen: 52
2026-04-19 08:57:30,522 [INFO] ✓ [sensor-ankara-01] → Kinesis | Temp: 24.47°C | Toplam gönderilen: 53
2026-04-19 08:57:30,721 [INFO] ✓ [sensor-ankara-02] → Kinesis | Temp: 24.87°C | Toplam gönderilen: 54
2026-04-19 08:57:30,907 [INFO] ✓ [sensor-ankara-03] → Kinesis | Temp: 25.95°C | Toplam gönderilen: 55
2026-04-19 08:57:31,091 [INFO] ✓ [sensor-ankara-04] → Kinesis | Temp: 24.56°C | Toplam gönderilen: 56
2026-04-19 08:57:32,581 [INFO] ✓ [sensor-ankara-01] → Kinesis | Temp: 24.59°C | Toplam gönderilen: 57
2026-04-19 08:57:32,793 [INFO] ✓ [sensor-ankara-02] → Kinesis | Temp: 24.9°C | Toplam gönderilen: 58
2026-04-19 08:57:32,980 [INFO] ✓ [sensor-ankara-03] → Kinesis | Temp: 25.32°C | Toplam gönderilen: 59
2026-04-19 08:57:33,827 [INFO] ✓ [sensor-ankara-04] → Kinesis | Temp: 25.13°C | Toplam gönderilen: 60
2026-04-19 08:57:34,583 [INFO] ✓ [sensor-ankara-01] → Kinesis | Temp: 24.62°C | Toplam gönderilen: 61
2026-04-19 08:57:34,824 [INFO] ✓ [sensor-ankara-02] → Kinesis | Temp: 25.04°C | Toplam gönderilen: 62

4. AWS Kinesis Data Streams ile Veri Akışı

kinesis_producer.py dosyası, WebSocket sunucusuna bağlanarak gelen veri akışını AWS Kinesis Data Streams'e iletir. Bu bileşen, WebSocket ile Kinesis arasında köprü görevi görür.

4.1 Kinesis Steam Konfigürasyonu

Parametre	Değer
Stream Adı	iot-sensor-stream
Kapasite Modu	On-Demand (Talebe Göre)
Bölge	us-east-1 (N. Virginia)
Partition Key	device id (sensör başına ayrı shard)

4.2 Producer Kodu

Her sensör kaydı, device_id değeri PartitionKey olarak kullanılarak Kinesis'e gönderilir. Bu sayede aynı sensörün verileri her zaman aynı shard üzerinden iletilir ve sıra garantisi sağlanır.

```
def send_to_kinesis(data: dict) -> bool:
    try:
        response = kinesis.put_record(
            StreamName=STREAM_NAME,
            Data=json.dumps(data).encode("utf-8"),
            PartitionKey=data["device_id"]
        )
```

5.AWS Lambda ile Veri İşleme

lambda_function.py, Kinesis Data Streams tarafından otomatik olarak tetiklenen bir AWS Lambda fonksiyonudur. Kinesis'ten gelen her batch (toplu kayıt grubu) için çalışır, veriyi işler, anomali kontrolü yapar ve DynamoDB'ye kaydeder.

The screenshot shows the AWS Lambda console for the 'iot-kinesis-processor' function. The function is configured to be triggered by Kinesis. The 'Kod kaynağı' (Code source) tab is selected, showing the 'lambda_function.py' file. The code is a Python Lambda function that processes data from Kinesis and stores it in DynamoDB.

The screenshot shows the code for the 'lambda_function.py' file. The code is a Python Lambda function that processes data from Kinesis and stores it in DynamoDB. The code includes imports for json, boto3, base64, logging, decimal, and datetime. It defines a 'float_to_decimal' function and a 'lambda_handler' function.

5.1. Anomali Tespiti

Lambda fonksiyonu, işlediği her kayıta basit eşik değeri kontrolü yaparak anomali tespiti gerçekleştirir. Tespit edilen anomaliler hem CloudWatch loglarına yazılır hem de DynamoDB kaydında işaretlenir.

Parametre	Eşik Değer	Uyarı Türü
Sıcaklık	> 30°C	Yüksek sıcaklık uyarısı
Nem	> %90	Aşırı nem uyarısı
CO2	> 1000 ppm	Tehlikeli CO2 seviyesi

5.2. DynamoDB Yazma İşlemi

Lambda fonksiyonu, Kinesis kayıtlarını base64 formatından çözerek JSON'a dönüştürür. DynamoDB, float veri tipini desteklemediğinden tüm ondalıklı sayılar Decimal tipine dönüştürülür.

```
def lambda_handler(event, context):  
    # 1. Base64 decode + JSON parse  
    raw = base64.b64decode(record["kinesis"]["data"])  
    data = json.loads(raw.decode("utf-8"))  
    # 2. Anomali kontrolü  
    anomaly_alerts = check_anomaly(data)  
    if anomaly_alerts:  
        alerts += 1  
        logger.warning(  
            f"ANOMALI [{device_id}]: {'', '.join(anomaly_alerts)}")  
    # 4. DynamoDB'ye yaz  
    table.put_item(Item=item)  
    processed += 1
```

6. AWS DynamoDB Veri Depolama

IoTSensorData tablosu, zaman serisi sensör verilerini depolamak üzere tasarlanmıştır. DynamoDB'nin NoSQL yapısı, şema esnekliği ve yüksek yazma performansı sayesinde gerçek zamanlı IoT uygulamaları için ideal bir seçimdir.

DynamoDB

Dashboard

Tables

Explore items

PartiQL editor

Backups

Exports to S3

Imports from S3

Integrations

Reserved capacity

Settings

DAX

Clusters

Subnet groups

Parameter groups

Events

Table: IoTSensorData - Items returned (50)

Scan started on April 19, 2026, 08:39:35

< 1 ... >

Actions Create item

☐	device_id (String)	timestamp (Number)	co2_ppm	has_alert	humidity	ingested_at	local
☐	sensor-ankara-01	1776514983	392.1	false	53.55	2026-04-18T...	Ank...
☐	sensor-ankara-01	1776514985	393.4	false	52.32	2026-04-18T...	Ank...
☐	sensor-ankara-01	1776514987	409.8	false	52.81	2026-04-18T...	Ank...
☐	sensor-ankara-01	1776514989	436.4	false	53.52	2026-04-18T...	Ank...
☐	sensor-ankara-01	1776514991	407.1	false	51.89	2026-04-18T...	Ank...
☐	sensor-ankara-01	1776514993	386	false	51.44	2026-04-18T...	Ank...
☐	sensor-ankara-01	1776514995	438.8	false	52.14	2026-04-18T...	Ank...
☐	sensor-ankara-01	1776514997	396.2	false	51.9	2026-04-18T...	Ank...
☐	sensor-ankara-01	1776514999	416.3	false	54.51	2026-04-18T...	Ank...
☐	sensor-ankara-01	1776515001	421.2	false	52.71	2026-04-18T...	Ank...
☐	sensor-ankara-01	1776515003	425.3	false	52.88	2026-04-18T...	Ank...

CloudShell

Feedback

Console Mobile App

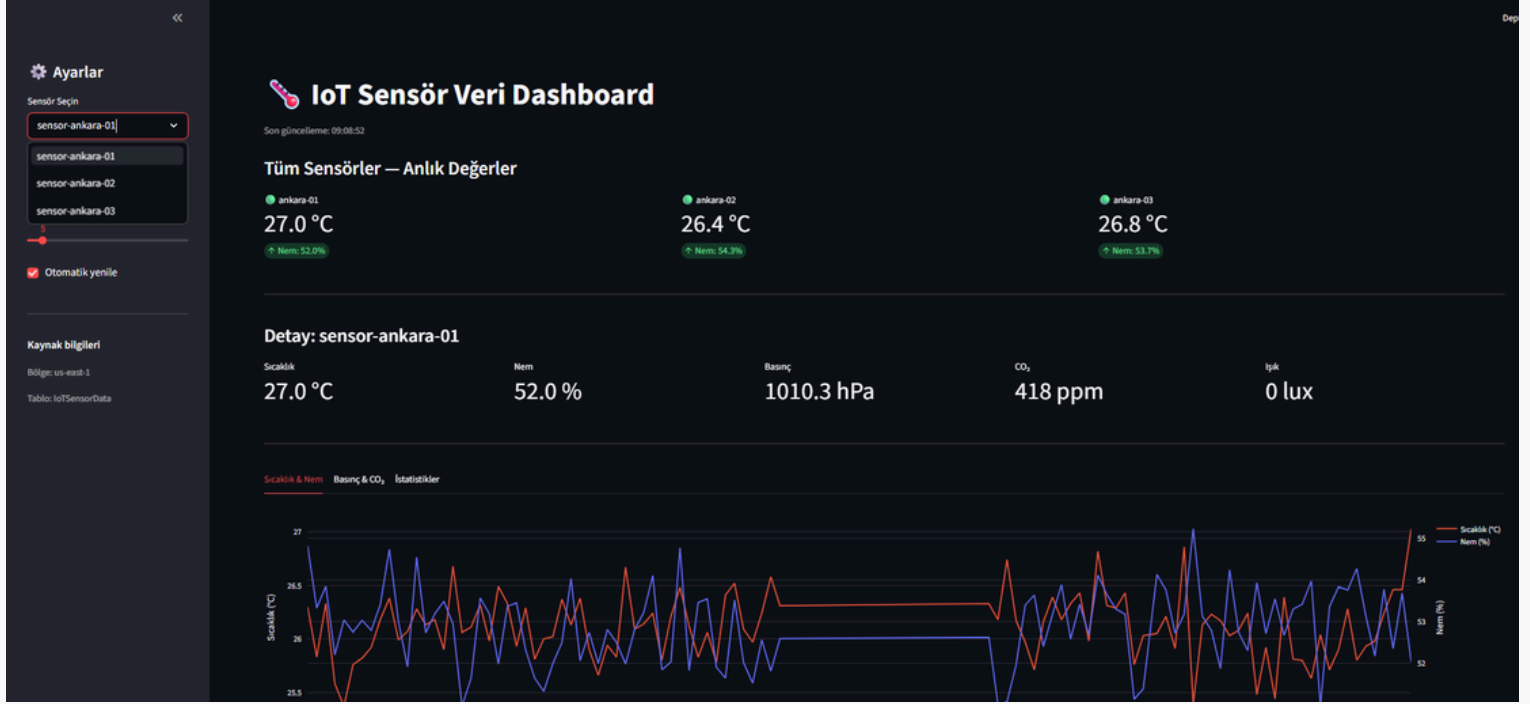
© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

7. Gerçek Zamanlı Dashboard

dashboard.py, Streamlit ve Plotly kütüphaneleri kullanılarak geliştirilmiş interaktif bir veri görselleştirme uygulamasıdır. DynamoDB'den çekilen veriler, anlık grafikler ve metrik kartlar aracılığıyla kullanıcıya sunulur.

7.1. Dashboard Özellikleri

- Tüm sensörler için anlık sıcaklık ve nem değerleri
- Seçili sensör için zaman serisi grafikleri (Sıcaklık & Nem, Basınç & CO2)
- İstatistiksel analiz (ortalama, min, max, standart sapma)
- Sıcaklık dağılım histogramı
- 5 saniyede bir otomatik yenileme
- Sol panel: sensör seçimi, kayıt sayısı ve yenileme süresi ayarları



8. AWS Bulut Altyapısı

Proje tamamen AWS'nin yönetilen servisleri üzerine inşa edilmiştir. aws_setup.py scripti, gerekli tüm AWS kaynaklarını otomatik olarak oluşturur.

8.1. AWS Servisleri

Servis	Kaynak Adı	Kullanım Amacı
Kinesis Data Streams	iot-sensor-stream	Gerçek zamanlı veri akışı
Lambda	iot-kinesis-processor	Sunucusuz veri işleme
DynamoDB	IoTSensorData	Zaman serisi veri depolama
IAM	iot-kinesis-processor-role	Lambda için erişim yetkilendirmesi
CloudWatch	Otomatik	Log izleme ve hata takibi

8.2. Veri Akışı Hiyerarşisi

Sistemin çalışma mantığı şu sırayı takip eder:

- Sensör simülatörü WebSocket üzerinden veri üretir.
- Kinesis Producer bu veriyi AWS Kinesis'e iletir.
- Lambda fonksiyonu Kinesis tetiklemesiyle otomatik çalışır, veriyi işler ve DynamoDB'ye yazar.
- Streamlit Dashboard DynamoDB'den okuyarak anlık görselleştirme yapar.

9. Sonuç ve Değerlendirme

Bu proje kapsamında, gerçek zamanlı IoT veri akışı sistemi başarıyla tasarlanmış ve hayata geçirilmiştir. Python ile geliştirilen sensör simülatörü, WebSocket protokolü üzerinden veri yayınlamış; bu veriler AWS Kinesis aracılığıyla buluta iletilmiş, Lambda fonksiyonu ile işlenmiş ve DynamoDB'de saklanmıştır.

Proje sürecinde AWS'nin yönetilen servisleri arasındaki entegrasyon deneyimi kazanılmıştır. Kinesis'in yüksek hacimli veri akışı yönetimindeki gücü, Lambda'nın sunucusuz mimarideki esnekliği ve DynamoDB'nin zaman serisi veri depolama kapasitesi uygulamalı olarak öğrenilmiştir.